# Отчёт по домашней работе № 3

«Технология MapReduce»

Морозов Дмитрий Александрович, ИСУ 466761

4 ноября 2026 г.

## 1 Развёртывание хранилища и загрузка данных

Для работы использован стенд из домашнего задания № 2: шесть каталогов дисков со схемой Erasure Coding **3+3**. После запуска все шесть дисков доступны. Каталоги расположены на одном физическом диске `D:`.

Я работал с вариантом, где вакансия была аналитики, поэтому всего у меня было 2 057 JSON-файлов. Первоначально провел небольшой анализ: В наборе 1 270 вакансий с навыками и 787 без навыков, 1 256 различных навыков, максимальный стек содержит 30 навыков. Зарплата указана у 579 вакансий.

Далее создал бакет `hh-lake`, а исходные JSON объединил в JSONL: одна строка соответствует одной вакансии. Исходные поля сохранил внутри `vacancy`, принадлежность варианту записана в `source_segment='analysts'`. Объект загружен по адресу:

`s3://hh-lake/raw/analysts/aa4d3566148d73b7/vacancies.jsonl`

После загрузки объект повторно прочитан из S3. Размер, число записей и SHA-256 совпали:


```
{
  "records": 2057,
  "bytes": 11858115,
  "local_sha256": "aa4d3566148d73b7accc2ed97b3a3c3a63f66c873e2ee4e6dc56dff5d548f9f6",
  "remote_sha256": "aa4d3566148d73b7accc2ed97b3a3c3a63f66c873e2ee4e6dc56dff5d548f9f6",
  "equal": true
}
```

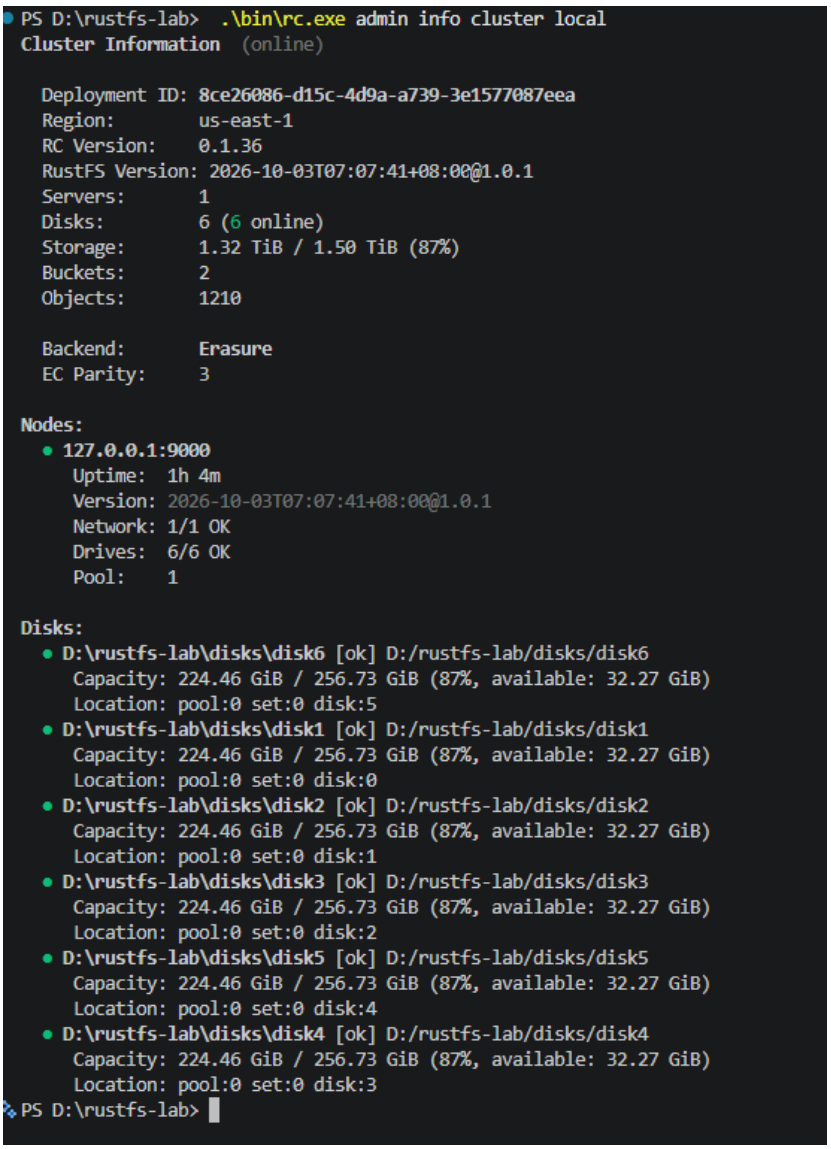



Рис. 1: Состояние RustFS: шесть дисков доступны, backend Erasure, EC parity = 3.

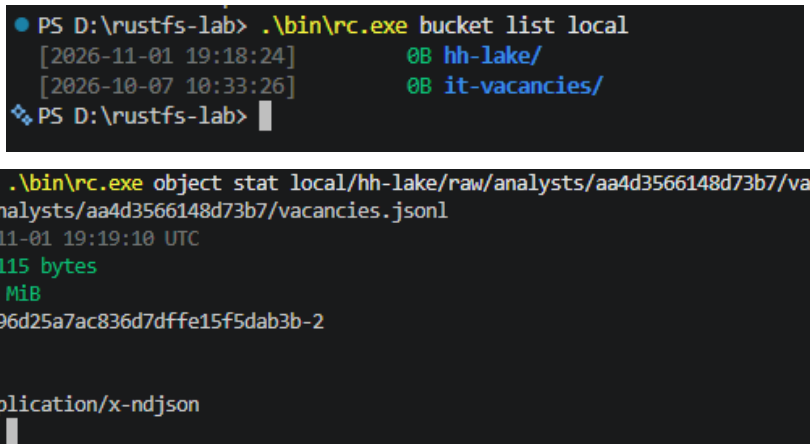



Рис. 2: Бакет hh-lake и загруженный объект vacancies.jsonl размером 11 858 115 байт.

# 2 Подсчёт навыков собственным MapReduce

Реализованы собственные Map, передача промежуточных результатов и Reduce на Python. Координатор читает объект из S3 чанками по 262 144 байт. Строки отделяются до декодирования UTF-8, поэтому границы чанков могут проходить внутри кириллического символа без повреждения JSON. Задание маппера содержит до 128 вакансий; работают два процесса, размер окна - не более четырёх заданий.

Навык учитывается один раз на вакансию. Пары и тройки представляют собой неупорядоченные сочетания разных навыков. Синонимы и регистр не объединяются. Для эксперимента выбраны три размера стека: первые 5, первые 10 и все уникальные навыки в исходном порядке. После ограничения навыки сортируются для канонической записи сочетаний. Вакансии без навыков сохраняются в общем числе входных записей.

Основная часть генерации ключей:

```
names = list(dict.fromkeys(
    s.strip() for s in record["vacancy"]["key_skills"] if s.strip()
))
names = sorted(names[:limit] if limit is not None else names)
for size in (1, 2, 3):
    for key in itertools.combinations(names, size):
        output.append((key, 1))
```

В таблице приведены суммы частот (число эмиссий) и число уникальных выходных ключей для одиночных навыков, пар и троек:

| Стек | Навыки | Пары | Тройки | Ключи 1 | Ключи 2 | Ключи 3 |
|---|---|---|---|---|---|---|
| 5 | 5 533 | 10 045 | 9 401 | 858 | 4 661 | 6 556 |
| 10 | 7 722 | 24 432 | 51 566 | 1 136 | 10 600 | 33 560 |
| Все | 8 810 | 40 778 | 180 490 | 1 256 | 15 647 | 84 815 |

Для вакансии с $m$ навыками число эмиссий равно

$$N_1 = m, \qquad N_2 = \binom{m}{2} = \frac{m(m-1)}{2}, \qquad N_3 = \binom{m}{3} = \frac{m(m-1)(m-2)}{6}.$$

При $m = 5$ формируется 10 пар и 10 троек, при $m = 10$ - 45 пар и 120 троек, при $m = 30$ - 435 пар и 4060 троек. По всему набору переход от пяти ко всем навыкам увеличил число одиночных эмиссий в 1,59 раза, пар - в 4,06 раза, троек - в 19,20 раза. Рост объясняется числом сочетаний внутри вакансии.

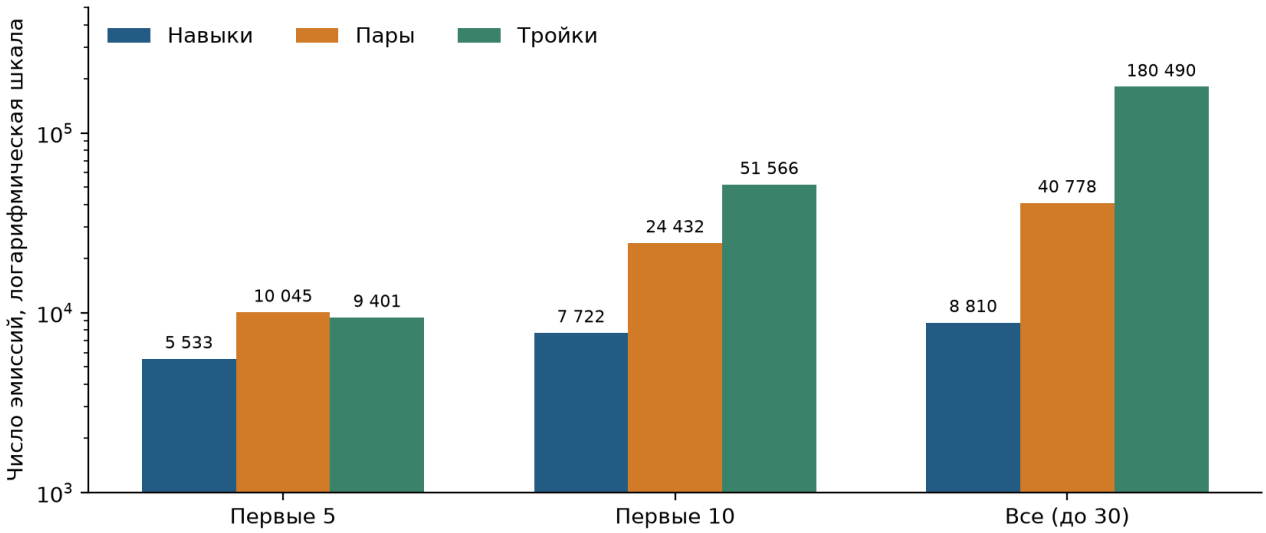



Рис. 3: Рост числа эмиссий при расширении стека навыков.

Полные результаты для трёх размеров стека сохранены в S3:

| Стек | Ключ результата внутри hh-lake | Размер, байт |
|---|---|---|
| 5 | `results/analysts/validation-20261101/results/mr5/counts.csv` | 1 042 869 |
| 10 | `results/analysts/validation-20261101/results/mr10/counts.csv` | 4 347 098 |
| Все | `results/analysts/validation-20261101/results/mrall/counts.csv` | 10 587 772 |

После загрузки каждый файл результата прочитан обратно из S3, контрольные суммы совпали с локальными файлами.

## 2.1 Проверка отказа маппера

При запуске полного стека с параметром `-fail-chunk 2` процесс маппера задания 2 принудительно завершается через `os._exit(70)`. Координатор обнаруживает закрытие канала и код завершения 70, создаёт новый процесс и повторяет задание. Окно вносится в Reduce после успешного завершения всех входящих в него заданий, поэтому повтор не приводит к двойному счёту.

Фрагмент журнала проверки:

```
{"event": "inject_mapper_exit", "chunk_id": 2, "exit_code": 70}
{"event": "mapper_process_failure", "chunk_id": 2, "attempt": 1, "exit_code": 70, "
    replay_chunks": [2]}
```

Штатный запуск и запуск с отказом содержат по 101 718 ключей; различий в ключах и частотах нет.

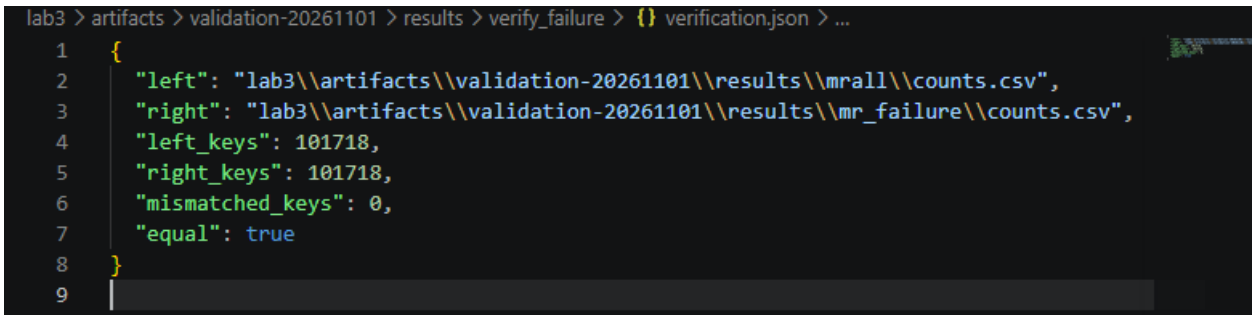



Рис. 4: Полное совпадение штатного результата и результата после отказа маппера.

# 3 Независимый расчёт DuckDB и верификация

DuckDB читает тот же JSONL напрямую из S3 через расширение `httpfs`. Условие индивидуального варианта написано в SQL:

```
FROM read_ndjson($uri, columns = {
  source_segment: 'VARCHAR',
  source_file: 'VARCHAR',
  vacancy: 'JSON'
})
WHERE source_segment = 'analysts'
```

SQL независимо разворачивает исходные навыки, удаляет повторы по первой позиции в списке, применяет ограничение стека и строит сочетания. Результаты или заранее подготовленные ключи MapReduce в запрос не передаются. Подсчёт выполняется следующим запросом:

```
-- Lexical comparisons enumerate each unordered combination once.
SELECT [skill] AS skills, count(*)::BIGINT AS frequency
FROM selected GROUP BY skill
UNION ALL
SELECT [a.skill, b.skill] AS skills, count(*)::BIGINT AS frequency
FROM selected a JOIN selected b ON a.vacancy_id = b.vacancy_id AND a.skill < b.skill
GROUP BY a.skill, b.skill
UNION ALL
SELECT [a.skill, b.skill, c.skill] AS skills, count(*)::BIGINT AS frequency
FROM selected a
JOIN selected b ON a.vacancy_id = b.vacancy_id AND a.skill < b.skill
JOIN selected c ON a.vacancy_id = c.vacancy_id AND b.skill < c.skill
GROUP BY a.skill, b.skill, c.skill;
```

Неравенства $a.skill < b.skill < c.skill$ исключают повторный учёт перестановок. Для каждого размера стека проверены все ключи и все частоты, а также входное число вакансий и суммы эмиссий:

| Стек | Ключей MapReduce | Ключей DuckDB | Расхождений |
|---|---:|---:|---:|
| 5 | 12 075 | 12 075 | 0 |
| 10 | 45 296 | 45 296 | 0 |
| Все | 101 718 | 101 718 | 0 |

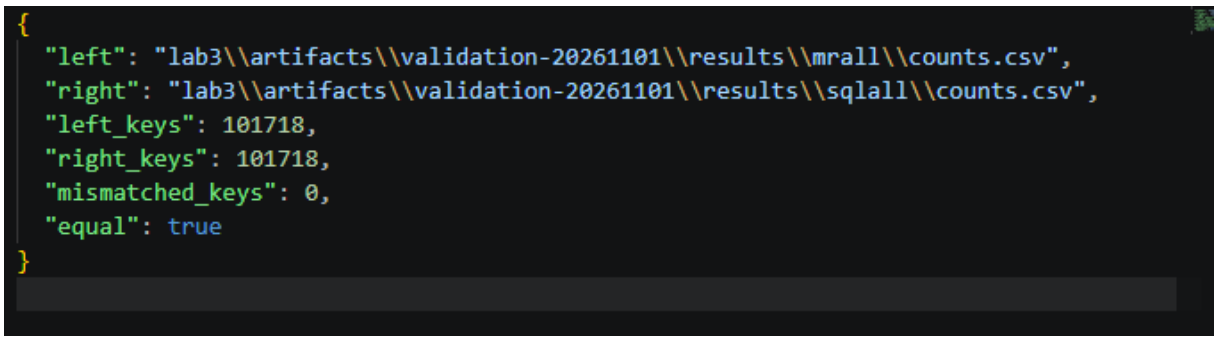



Рис. 5: Совпадение всех 101 718 ключей и частот для полного стека.

Таким образом, контрольные результаты DuckDB и MapReduce совпали для одиночных навыков, пар и троек во всех трёх режимах.

# 4 Связь навыков с заработной платой

Зарплаты в рублях «на руки» (`gross=false`) и до налогов (`gross=true`) анализируются раздельно. В наборе соответственно 378 и 197 таких вакансий; навыки указаны у 247 и 133 из них. Четыре зарплаты в USD/EUR исключены из рублёвого сравнения. Пропущенные суммы не заменяются нулями.

Если указаны обе границы, используется середина вилки: $s = (s_{\min} + s_{\max})/2$; при одной границе - её значение. Медиана рассчитывается по этим представителям зарплат. Для пары базой служит более частый из её навыков по всем вакансиям, для тройки - наиболее частая входящая пара. Базовая группа включает вакансии с полным сочетанием. Прирост определяется как

$$\Delta = \operatorname{median}(s \mid S) - \operatorname{median}(s \mid B), \qquad \Delta_{\%} = 100\frac{\Delta}{\operatorname{median}(s \mid B)}.$$

Минимальная поддержка рейтинга - пять зарплатных наблюдений. В таблицах $n$ - число зарплат сочетания, $n_0$ - число зарплат базы. Показаны по два наибольших положительных и отрицательных значения $\Delta$, отдельно для пар и троек. Суммы указаны в рублях «на руки».

## 4.1 Пары навыков

| Сочетание | База | $n/n_0$ | Медиана, руб. | База, руб. | $\Delta$, руб. |
|---|---|---|---|---|---|
| REST + SQL | SQL | 6/57 | 292 500 | 130 000 | 162 500 |
| ERP-системы на базе 1С + Аналитическое мышление | Аналитическое мышление | 5/107 | 235 000 | 90 000 | 145 000 |
| 1С: Предприятие 8 + ERP-системы на базе 1С | ERP-системы на базе 1С | 5/21 | 125 000 | 220 000 | −95 000 |
| ERP-системы на базе 1С + Разработка инструкций | ERP-системы на базе 1С | 6/21 | 137 500 | 220 000 | −82 500 |

Наибольшая положительная разница у REST + SQL относительно SQL: 292 500 против 130 000 руб., то есть +162 500 руб. (+125%), $n = 6$. Однако три из шести предложений имеют только верхнюю границу; на трёх полных вилках медиана сочетания равна

135 000 руб., а прирост относительно полной-вилочной базы SQL - 21 275 руб. Большой исходный эффект чувствителен к способу представления вилок.

Отрицательная разница у «1С: Предприятие 8 + ERP-системы на базе 1С» составляет −95 000 руб. В сочетании три из пяти зарплат относятся к опыту 1–3 года. Для опыта 3–6 лет медианы сочетания и базы равны 242 500 и 237 500 руб.; внутри этой небольшой группы разница положительна. Следовательно, часть общего снижения может объясняться различиями состава вакансий по опыту.

## 4.2 Тройки навыков

| Сочетание | База | $n/n_0$ | Медиана, руб. | База, руб. | Δ, руб. |
|---|---|---|---|---|---|
| REST + SQL + UML | SQL + UML | 5/12 | 285 000 | 162 500 | 122 500 |
| ERP-системы на базе 1С + Постановка задач разработчикам + Разработка технических заданий | Постановка задач разработчикам + Разработка технических заданий | 7/19 | 250 000 | 150 000 | 100 000 |
| BPMN + REST + SOAP | REST + SOAP | 5/10 | 135 000 | 282 500 | −147 500 |
| ERP-системы на базе 1С + Разработка инструкций + Разработка технических заданий | ERP-системы на базе 1С + Разработка технических заданий | 5/11 | 125 000 | 240 000 | −115 000 |

Для REST + SQL + UML относительно SQL + UML разница равна +122 500 руб., $n = 5$; на полных вилках она меняет знак: 135 000 против 155 000 руб., $n = 3$. Для BPMN + REST + SOAP относительно REST + SOAP наблюдается −147 500 руб.; на полных вилках разница уменьшается до −27 500 руб. Эти примеры показывают влияние малой поддержки и односторонних зарплатных границ на лидеров рейтинга.

## 4.3 Проверка устойчивости

При повышении порога с 5 до 10 зарплатных наблюдений число допустимых пар сокращается с 203 до 46, троек - со 115 до 5. Примеры на более строгом пороге:

| Сочетание | База | $n/n_0$ | Медиана, руб. | База, руб. | Δ, руб. |
|---|---|---|---|---|---|
| ERP-системы на базе 1С + Разработка технических заданий | Разработка технических заданий | 11/38 | 240 000 | 126 000 | 114 000 |
| Аналитическое мышление + Системный анализ | Аналитическое мышление | 12/107 | 192 500 | 90 000 | 102 500 |
| Бизнес-анализ + Системный анализ | Системный анализ | 13/30 | 150 000 | 210 000 | −60 000 |
| BPMN + Аналитика | BPMN | 10/36 | 82 500 | 140 000 | −57 500 |

| Сочетание | База | $n/n_0$ | Медиана, руб. | База, руб. | Δ, руб. |
|---|---|---|---|---|---|
| Аналитика + Аналитическое мышление + Бизнес-анализ | Аналитическое мышление + Бизнес-анализ | 12/19 | 115 000 | 100 000 | 15 000 |
| Анализ данных + Аналитическое мышление + Сбор и анализ информации | Анализ данных + Аналитическое мышление | 12/30 | 75 000 | 72 500 | 2 500 |
| Аналитика + Аналитическое мышление + Работа с большим объемом информации | Аналитическое мышление + Работа с большим объемом информации | 10/28 | 77 500 | 85 000 | −7 500 |

У пар остаётся положительная разница 114 000 руб. для «ERP-системы на базе 1С + Разработка технических заданий», $n = 11$, и отрицательная 60 000 руб. для «Бизнес-анализ + Системный анализ», $n = 13$. У троек крупные исходные эффекты не сохраняются: среди приведённых примеров положительная разница равна 15 000 руб., отрицательная - 7 500 руб.

Для зарплат до налогов сформированы отдельные рейтинги. В этой группе 17 вакансий со стеком не менее 25 навыков относятся к одному работодателю и имеют вилку 70 000–80 000 руб. Повторяющиеся объявления увеличивают частоты многих сочетаний. Интерпретация разностей медиан поэтому учитывает поддержку, опыт, регион, полноту вилки и состав работодателей.

# 5 Региональный анализ

Сравниваются Москва, Санкт-Петербург и Екатеринбург по полю `area` исходных вакансий. Все навыки используются без ограничения стека. Региональные медианы включают вакансии без навыков, если у них указана подходящая зарплата.

**Таблица 1. Общие показатели регионов**

| Регион | Всего | С навыками | $n_{\text{net}}$ | Медиана net | $n_{\text{gross}}$ | Медиана gross |
|---|---|---|---|---|---|---|
| Москва | 988 | 595 | 142 | 170 000 | 46 | 133 500 |
| Санкт-Петербург | 251 | 168 | 43 | 135 000 | 21 | 105 000 |
| Екатеринбург | 60 | 38 | 18 | 92 500 | 5 | 67 500 |

Зарплаты net приведены «на руки», gross - до налогов. Объёмы зарплатных выборок показаны отдельно для каждой группы.

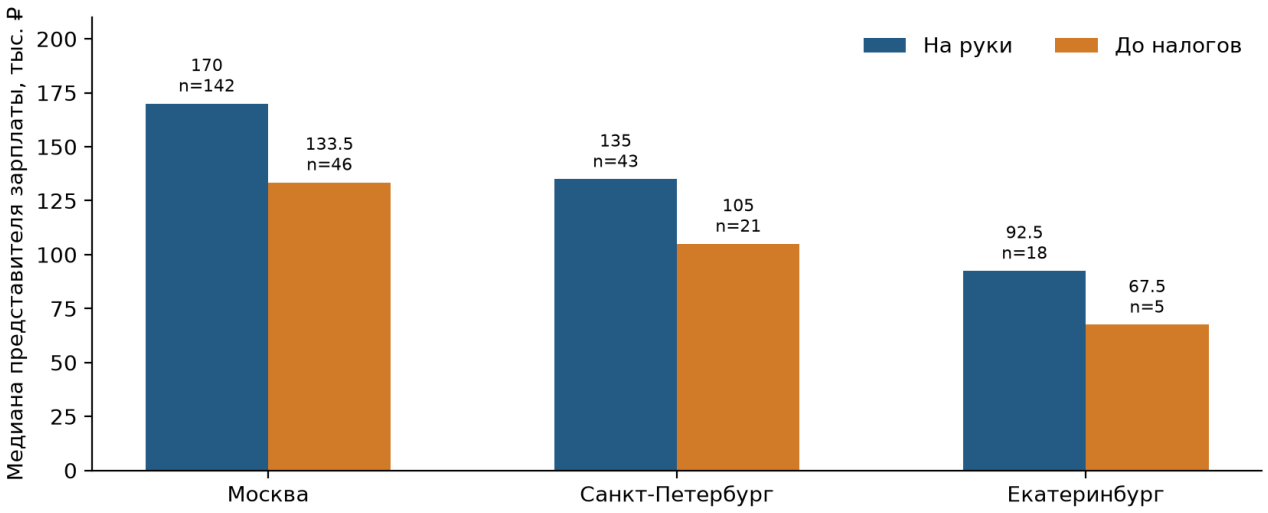



Рис. 6: Медианы зарплат и размеры выборок по трём регионам.

Медиана net в Москве (170 000 руб.) выше петербургской (135 000 руб.) на 25,9%, екатеринбургской (92 500 руб.) - на 83,8%. При этом доля вакансий с указанной net-зарплатой составляет 14,4%, 17,1% и 30,0% соответственно. Различия заполненности зарплат учитываются при сравнении регионов.

## Таблица 2. Востребованность навыков, пар и троек

В таблице показан один лидер каждого размера для каждого региона. При равенстве частот используется лексикографический выбор. Доля рассчитана от всех вакансий региона.

| Регион | Размер | Навык / сочетание | Частота | Доля | $n_{\text{net}}$ | Медиана net |
|---|---|---|---|---|---|---|
| Москва | 1 | SQL | 238 | 24,1% | 25 | 155 000 |
| Москва | 2 | BPMN + UML | 92 | 9,3% | 12 | 237 500 |
| Москва | 3 | BPMN + SQL + UML | 54 | 5,5% | 3 | 330 000 |
| Санкт-Петербург | 1 | SQL | 59 | 23,5% | 9 | 150 000 |
| Санкт-Петербург | 2 | SQL + Системный анализ | 22 | 8,8% | 2 | 172 500 |
| Санкт-Петербург | 3 | REST + SQL + Системный анализ | 10 | 4,0% | 1 | 285 000 |
| Екатеринбург | 1 | Аналитическое мышление | 14 | 23,3% | 10 | 92 500 |
| Екатеринбург | 2 | Анализ данных + Аналитическое мышление | 4 | 6,7% | 3 | 85 000 |
| Екатеринбург | 3 | BPMN + Бизнес-анализ + Постановка задач разработчикам | 3 | 5,0% | 0 | — |

SQL встречается в 238 вакансиях Москвы (24,1%) и 59 вакансиях Санкт-Петербурга (23,5%): доли близки, хотя абсолютные частоты отличаются примерно в четыре раза. В Москве наиболее частая пара BPMN + UML имеет 92 вакансии; тройка BPMN + SQL + UML - 54. В Санкт-Петербурге пара SQL + Системный анализ имеет 22 вакансии, тройка REST + SQL + Системный анализ - 10.

В Екатеринбурге лидирует «Аналитическое мышление»: 14 вакансий (23,3%). Максимальная частота пары равна четырём, тройки - трём. Зарплатных наблюдений для таких сочетаний мало: у показанной тройки net-зарплаты отсутствуют. Региональные выводы по оплате сочетаний здесь ограничены объёмом данных.

# 6 Сравнение производительности и оптимизация

Среда измерений: Windows, Python 3.11.9, DuckDB 2.0.0, 12 логических CPU, 15,7 GiB RAM. У MapReduce два процесса мапперов, у DuckDB два потока. Для каждого метода и размера стека выполнены три независимых запуска в новых процессах, всего 27 попыток с учётом оптимизированного MapReduce.

Время расчёта включает чтение S3 и получение частот. Wall time дополнительно включает запуск процесса, импорты, настройку клиента и экспорт CSV. Пик памяти - максимум суммы RSS всего дерева клиентских процессов, измеряемой внешним `psutil` каждые 10 мс. RustFS в клиентскую память не включён. Кеши ОС и сервера принудительно не очищались; порядок методов менялся между повторами.

В таблице приведены медианы трёх попыток:

| Стек | Метод | Расчёт, с | Wall time, с | Пик RSS, MiB |
|---|---|---|---|---|
| 5 | MR | 0,337 | 1,207 | 123,40 |
| 5 | DuckDB | 0,141 | 0,633 | 141,02 |
| 5 | MR + combiner | 0,326 | 1,066 | 121,73 |
| 10 | MR | 0,411 | 1,425 | 125,00 |
| 10 | DuckDB | 0,249 | 0,918 | 143,91 |
| 10 | MR + combiner | 0,375 | 1,334 | 124,64 |
| Все | MR | 0,486 | 1,643 | 142,10 |
| Все | DuckDB | 0,431 | 1,475 | 144,76 |
| Все | MR + combiner | 0,505 | 1,608 | 133,48 |

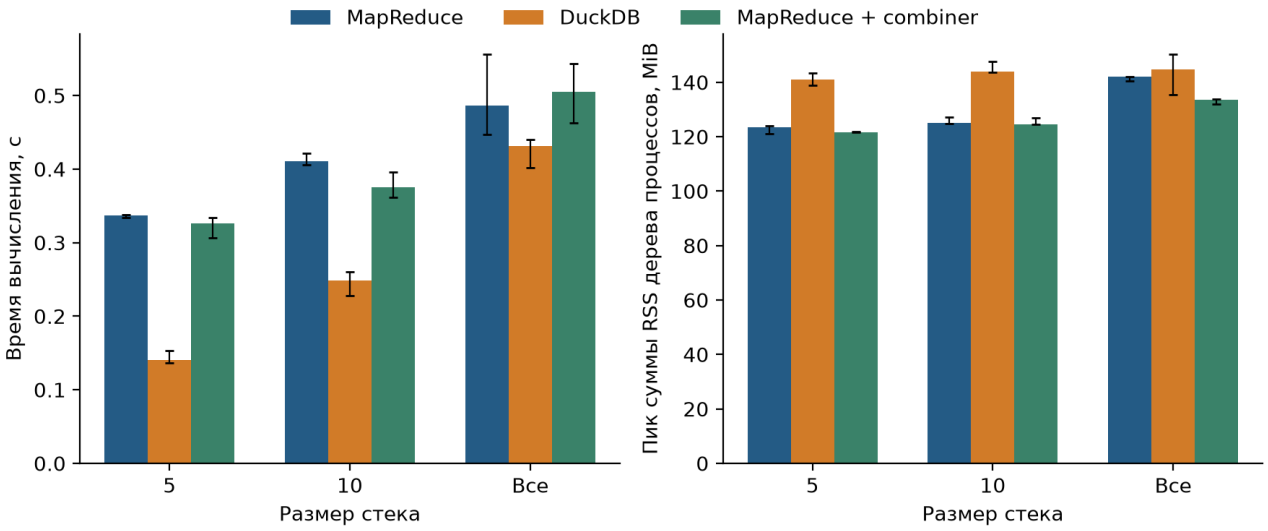



Рис. 7: Медианы времени расчёта и пика RSS; интервалы показывают минимум и максимум трёх попыток.

DuckDB быстрее базового MapReduce для всех трёх размеров стека. На полном стеке медиана wall time равна 1,475 с против 1,643 с. Для небольших стеков заметна постоянная стоимость запуска Python-процессов; по мере роста числа сочетаний разрыв сокращается. Пики RSS полного стека сопоставимы: 144,76 MiB у DuckDB и 142,10 MiB у базового MapReduce. Удержание всех уникальных ключей Reduce определяет рост памяти MapReduce вместе с размером результата.

## 6.1 Бонус: локальное суммирование у мапперов

В базовом варианте маппер передаёт каждую эмиссию `(key, 1)`. Оптимизация суммирует одинаковые ключи внутри задания через `Counter` до передачи координатору. Остальные параметры сохранены. Для полного стека получено:

| Показатель | До | После |
|---|---|---|
| Записей shuffle | 230 078 | 163 525 |
| Медиана пика RSS, MiB | 142,10 | 133,48 |
| Медиана времени расчёта, с | 0,486 | 0,505 |
| Медиана wall time, с | 1,643 | 1,608 |

Число переданных записей сократилось на 28,9%, медианный пик RSS - на 6,1%. Медиана времени расчёта выросла на 3,8%; медиана wall time уменьшилась на 2,2%. Диапазоны времени перекрываются, поэтому устойчивого ускорения полного стека по трём попыткам не установлено. Подтверждённое улучшение - сокращение shuffle и памяти. Результаты всех 27 запусков совпали с контрольными таблицами по SHA-256.

### Результаты трёх повторов

Таблица 1: Все измеряемые запуски: время и пик памяти клиентского дерева процессов.

| Стек | Метод | Попытка | Расчёт, с | Wall time, с | Пик RSS, MiB |
|---|---|---|---|---|---|
| 5 | MR | 1 | 0,338 | 1,136 | 120,98 |
| 5 | DuckDB | 1 | 0,141 | 0,633 | 141,02 |
| 5 | MR + combiner | 1 | 0,307 | 0,987 | 121,80 |
| 5 | DuckDB | 2 | 0,137 | 0,574 | 138,96 |
| 5 | MR + combiner | 2 | 0,326 | 1,066 | 121,59 |
| 5 | MR | 2 | 0,334 | 1,207 | 123,90 |
| 5 | MR + combiner | 3 | 0,334 | 1,169 | 121,73 |
| 5 | MR | 3 | 0,337 | 1,243 | 123,40 |
| 5 | DuckDB | 3 | 0,153 | 0,664 | 143,47 |
| 10 | MR | 1 | 0,421 | 1,434 | 127,14 |
| 10 | DuckDB | 1 | 0,249 | 0,918 | 143,71 |
| 10 | MR + combiner | 1 | 0,361 | 1,334 | 124,52 |
| 10 | DuckDB | 2 | 0,228 | 0,907 | 143,91 |
| 10 | MR + combiner | 2 | 0,396 | 1,476 | 127,04 |
| 10 | MR | 2 | 0,411 | 1,425 | 125,00 |
| 10 | MR + combiner | 3 | 0,375 | 1,311 | 124,64 |
| 10 | MR | 3 | 0,406 | 1,350 | 124,71 |
| 10 | DuckDB | 3 | 0,260 | 0,921 | 147,57 |
| Все | MR | 1 | 0,447 | 1,643 | 142,10 |
| Все | DuckDB | 1 | 0,402 | 1,409 | 150,22 |
| Все | MR + combiner | 1 | 0,462 | 1,608 | 131,95 |
| Все | DuckDB | 2 | 0,431 | 1,475 | 144,76 |
| Все | MR + combiner | 2 | 0,505 | 1,591 | 133,48 |
| Все | MR | 2 | 0,486 | 1,640 | 142,14 |
| Все | MR + combiner | 3 | 0,543 | 1,842 | 133,82 |
| Все | MR | 3 | 0,556 | 1,785 | 140,35 |
| Все | DuckDB | 3 | 0,441 | 1,599 | 135,53 |

# 7 Выводы

Собственный MapReduce и независимый SQL-запрос DuckDB дали одинаковые частоты навыков, пар и троек для варианта «аналитики» при всех трёх размерах стека. Проверка отказа маппера подтвердила корректность повторного выполнения без двойного счёта. Комбинаторный рост троек оказался существенно сильнее роста одиночных навыков.

Зарплатный анализ выявил сочетания с положительными и отрицательными разностями медиан; максимальные различия при поддержке 5–6 наблюдений оказались чувствительны к полноте зарплатных вилок. Региональный анализ показал близкую относительную востребованность SQL в Москве и Санкт-Петербурге и ограниченность зарплатных выводов по сочетаниям в Екатеринбурге.

Профилирование подтвердило преимущество DuckDB по времени на этом наборе. Оптимизация MapReduce локальным суммированием уменьшила объём передачи проме-

жуточных данных и пик памяти при сохранении точных результатов.

# 8 Ссылка на репо

Исходный код и результаты работы: `https://github.com/dmitmorozov/data-engineering-lab3`.